



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

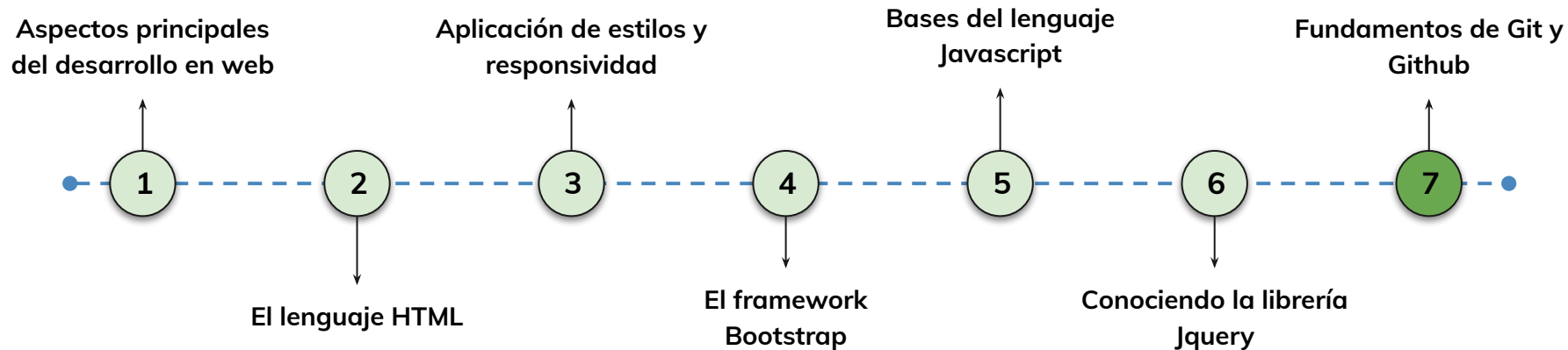


› Fundamentos de git y github - Parte II

AE 7.2: GESTIONAR EL CÓDIGO FUENTE DE UN PROYECTO UTILIZANDO GITHUB PARA MANTENER UN REPOSITORIO DE CÓDIGO REMOTO SEGURO Y PERMITIR TRABAJO CONCURRENTES.

Hoja de ruta

¿Cuáles skills conforman el programa?



Learning Path

¿Cuáles temas trabajaremos hoy?

17.

Git y Github

En esta sesión, profundizaremos en el uso avanzado de Git y GitHub, enfocándonos en el trabajo con repositorios remotos, Pull Requests y colaboración en equipo.

Repositorios Remotos y Colaboración

Uso de git push y git pull para sincronizar cambios con un repositorio remoto.

Diferencia entre git fetch y git pull y su aplicación en proyectos colaborativos.

Pull Requests y Versionado

Creación y gestión de Pull Requests (PRs) en GitHub para revisión de código.

Uso de tags y versiones (git tag, git push --tags)



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender los fundamentos de GitHub como una plataforma de gestión de control de versiones.
- Dominar la gestión de repositorios remotos y la sincronización de cambios mediante Push y Pull.
- Distinguir entre Fetch y Pull y aplicar el concepto apropiado según la situación.
- Aprender a clonar un repositorio existente para colaborar en proyectos.
- Adquirir habilidades para administrar Pull Requests y revisar y fusionar cambios de colaboradores.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- *Introducción a Git y GitHub: Definición, ventajas y diferencias entre ambos.*
- *Instalación y configuración de Git: Configuración de usuario y primeros comandos.*
- *Comandos básicos de Git: init, clone, add, commit, push.*
- *Manejo de ramas: Creación, cambio y fusión de ramas (branch, checkout, merge).*

Trabajando con Git y Github

< > Repositorios remotos

Push y Pull Fetch v/s Pull

Los repositorios remotos, y los comandos push, pull, fetch y pull son conceptos y comandos importantes en Git y GitHub para colaborar con otros desarrolladores y mantener sincronizados los cambios en un proyecto.



Uso de Repositorios Remotos en GitHub

¿En qué consistirá la Demo?

El objetivo es poder generar el primer repositorio de trabajo.

1. Crear un repositorio en GitHub.
2. Usar `git clone URL_REPO` para traer una copia del repositorio a la máquina local.
3. Editar un archivo en el proyecto.
4. Preparar los cambios para subirlos al repositorio remoto
5. Subir los cambios al repositorio remoto
6. Obtener cambios desde el repositorio remoto
7. Crear una nueva rama para agregar cambios y genera algun cambio dentro
8. Cambiar de rama y subir la nueva rama a github
9. Crear un Pull Request y fusionar cambios

Tiempo: 25 minutos

< > Repositorios remotos

Push y Pull Fetch v/s Pull

Repositorios remotos:

Un repositorio remoto no es más que una copia del repositorio Git en un servidor remoto, como GitHub, GitLab o Bitbucket.

El cual permite colaborar con otros desarrolladores, compartir el código y realizar un seguimiento de los cambios en un proyecto de manera centralizada.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

Repositorios remotos:

Una gran ventaja es que se puede clonar un repositorio remoto en tu máquina local utilizando el comando git clone.

```
git clone https://github.com/usuario/turepo.git
```

Recorda que debes cambiar `https://github.com/usuario/turepo.git` por la url de tu proyecto.

Después de clonar, tendrás una copia local del repositorio y podrás realizar cambios y commits en tu máquina.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

push:

El comando push se utiliza para enviar los commits locales al repositorio remoto. Lo que permite compartir tus cambios con otros desarrolladores y mantener sincronizado el repositorio remoto con tus últimas modificaciones.

Utiliza el comando `git push` seguido del nombre de la rama y el repositorio remoto al que deseas enviar los cambios.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

push:

arrancamos clonando nuestro repositorio, utilizaremos push para que los cambios realizados se guarden en el repositorio remoto. Pero ahora, además de clonar, vamos a crearle algunos cambios, para que sean guardados utilizando push

- Ya tenemos esto, del paso anterior:

```
git clone https://github.com/usuario/turepo.git
```

- Puedes realizar cambios en el archivo:

Navega al directorio del repositorio clonado y realiza los cambios deseados en un archivo específico utilizando el editor de texto o el IDE de tu elección.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

push:

- Verificar el estado de los cambios:

Utiliza el comando `git status` para verificar el estado de los cambios en tu repositorio local. Deberías ver el archivo modificado listado como un archivo sin seguimiento.

- Agregar los cambios al área de preparación:

Utiliza el comando `git add` seguido del nombre del archivo modificado para agregar los cambios al área de preparación (staging area). Por ejemplo:

```
git add nombre_del_archivo_modificado
```



< > Repositorios remotos

Push y Pull Fetch v/s Pull

push:

- Realizar el commit:

Realiza un commit para guardar los cambios en el repositorio local.

Utiliza el comando `git commit -m` seguido de un mensaje descriptivo para el commit.

```
git commit -m "Agregados cambios en el archivo"
```



< > Repositorios remotos

Push y Pull Fetch v/s Pull

push:

- Realizar el push:

Utiliza el comando git push seguido del nombre del repositorio remoto y la rama a la que deseas enviar tus cambios.

```
git push origin master
```

Reemplaza origin con el nombre del repositorio remoto (generalmente es origin si clonaste el repositorio) y master con el nombre de la rama en la que deseas realizar el push.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

push:

- Opcional:

Si es la primera vez que haces un push en esta rama, es posible que necesites agregar la opción -u para establecer la rama upstream:

```
git push -u origin master
```



< > Repositorios remotos

Push y Pull Fetch v/s Pull

push:

→ Proporciona tus credenciales:

Si es la primera vez que realizas un push a este repositorio remoto o si se requiere autenticación, se te solicitará ingresar tus credenciales de GitHub (nombre de usuario y contraseña o token de acceso personal).

→ Espera a que finalice el push:

Git enviará tus cambios al repositorio remoto en GitHub. Una vez que el push se complete, verás un mensaje indicando que los cambios se han enviado correctamente.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

pull:

El comando pull se utiliza para recuperar los cambios del repositorio remoto y fusionarlos con tu copia local. Este comando se combina con el comando **fetch** (lo veremos a continuación) que es para recuperar y con el comando **merge** que es para fusionar (este ya lo trabajamos anteriormente) en un solo paso.

Utiliza el comando **git pull** | seguido del nombre de la rama y el repositorio remoto desde donde deseas obtener los cambios.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

pull:

→ **Verificar el estado de tus cambios locales:**

Antes de realizar un pull, es recomendable verificar el estado de tus cambios locales. Utiliza el comando `git status` para asegurarte de que no tengas cambios sin confirmar.

→ **Realizar un pull desde el repositorio remoto:**

Utiliza el comando `git pull` seguido del nombre del repositorio remoto y la rama de la cual deseas obtener los cambios.

```
git pull origin master
```

Reemplaza `origin` con el nombre del repositorio remoto (generalmente es `origin` si clonamos el repositorio) y `master` con el nombre de la rama desde la cual deseas obtener los cambios.



< > Repositorios remotos

Push y Pull Fetch v/s Pull

pull:

→ **Proporcionar sus credenciales:**

Si se requiere autenticación para acceder al repositorio remoto, es posible que se te solicite ingresar tus credenciales de GitHub (nombre de usuario y contraseña o token de acceso personal).

→ **Esperar a que finalice el pull:**

Git recuperará los últimos cambios del repositorio remoto y los fusionará automáticamente en tu rama local. Si no hay conflictos, el pull se completará sin problemas.

En caso de que haya conflictos, Git te notificará y mostrará los archivos afectados. Deberás resolver los conflictos manualmente editando los archivos en conflicto y luego hacer un commit para finalizar la fusión.

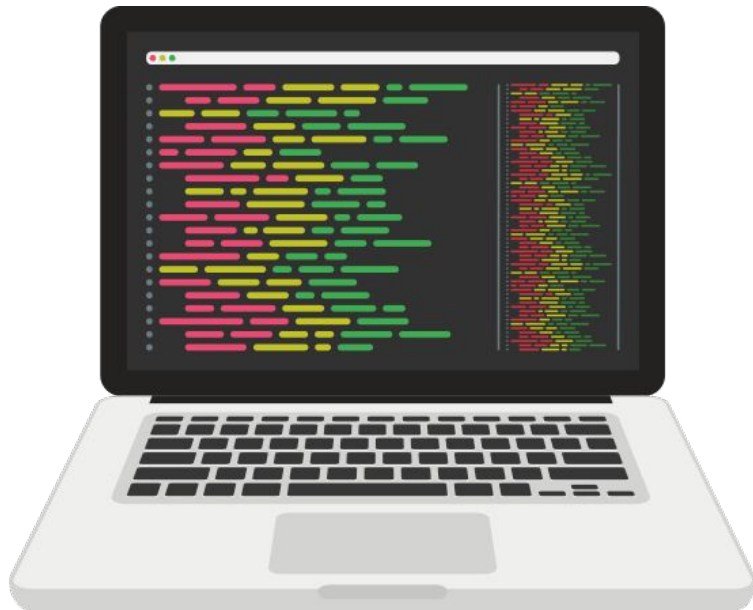


< > fetch vs pull en GitHub

En GitHub, fetch se utiliza para obtener los cambios del repositorio remoto sin fusionar automáticamente los cambios en tu rama local.

Con fetch, los cambios se descargan a tu repositorio local, pero no se fusionan automáticamente, lo que te permite revisar los cambios antes de incorporarlos.

Para fusionar los cambios después de realizar fetch, puedes utilizar el comando `git merge` o `git rebase` para combinar los cambios descargados con tu rama local.



< > fetch vs pull en GitHub

Por otro lado, pull en GitHub es similar a fetch pero automáticamente fusiona los cambios descargados en tu rama local sin necesidad de ejecutar un comando de fusión adicional.

Si estás trabajando en un proyecto en solitario o en una rama dedicada, pull puede ser conveniente.

Sin embargo, si estás colaborando con otros desarrolladores y deseas revisar los cambios antes de fusionarlos, puedes optar por fetch y luego realizar la fusión manualmente.



fetch

en GitHub

Fetch:

- Cuando realizas un "Fetch", obtienes una copia de los cambios del repositorio remoto en tu repositorio local.
- Los cambios se descargan y se almacenan en tu repositorio local, pero no se aplican automáticamente a tu rama de trabajo actual.
- Puedes inspeccionar los cambios descargados antes de decidir fusionarlos en tu rama actual.

pull

en GitHub

Pull:

- Un "Pull" es una operación que combina dos pasos en uno: primero, obtiene los cambios del repositorio remoto (realiza un "Fetch") y luego los fusiona automáticamente en tu rama de trabajo actual.
- Es una forma más conveniente de actualizar tu rama local con los cambios del repositorio remoto en comparación con un "Fetch" seguido de una fusión manual.



Momento:

Time-out!



5 -10 min.

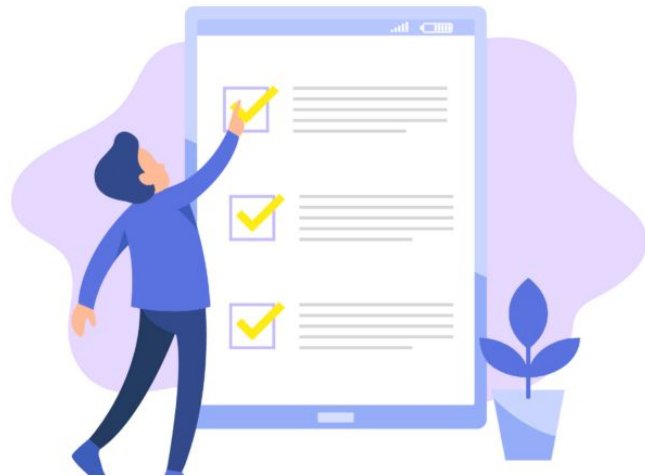


< > Administrando Pull Request

¿Qué es un pull request?

Un **pull request** o **solicitud de extracción** es una función clave en plataformas de alojamiento de repositorios como GitHub. Es un mecanismo que permite a los desarrolladores colaborar y revisar los cambios realizados en un repositorio antes de que sean fusionados en la rama principal o en otra rama.

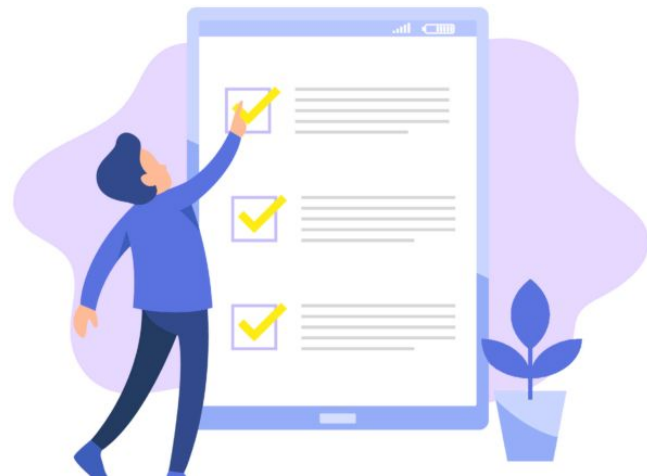
Podemos verlo como una solicitud que un desarrollador envía al propietario o colaboradores del repositorio para que revisen y consideren los cambios propuestos.



< > Administrando Pull Request

El flujo típico de trabajo con pull requests:

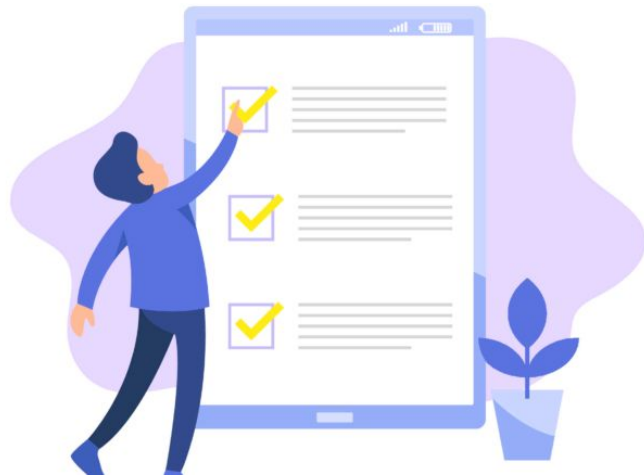
- Un desarrollador crea una rama separada en el repositorio para trabajar en una nueva función, solucionar un error o realizar cualquier otro tipo de modificación.
- Después de realizar los cambios y realizar los commits en su rama, el desarrollador envía un pull request al repositorio principal o a la rama base a la que desea fusionar sus cambios.



< > Administrando Pull Request

El flujo típico de trabajo con pull requests:

- Otros colaboradores pueden revisar el pull request, ver los cambios realizados, hacer comentarios y sugerencias, y solicitar modificaciones adicionales si es necesario.
- El autor del pull request puede realizar las modificaciones solicitadas, agregar más commits a su rama y actualizar automáticamente el pull request para que los revisores vean los cambios actualizados.

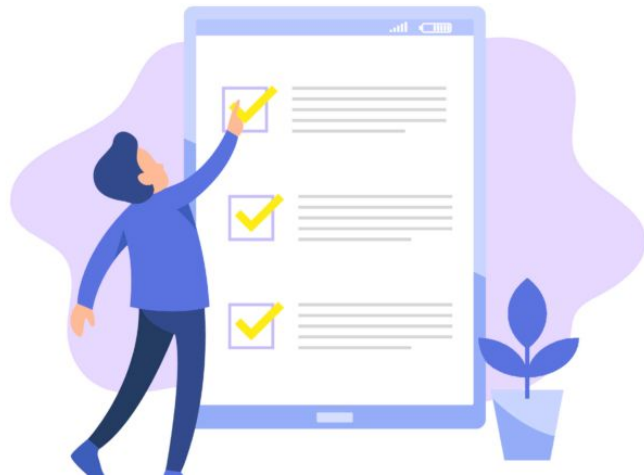


< > Administrando Pull Request

El flujo típico de trabajo con pull requests:

- Una vez que los cambios en el pull request han sido aprobados y revisados adecuadamente, el propietario del repositorio o un colaborador con permisos suficientes puede fusionar los cambios en la rama base.

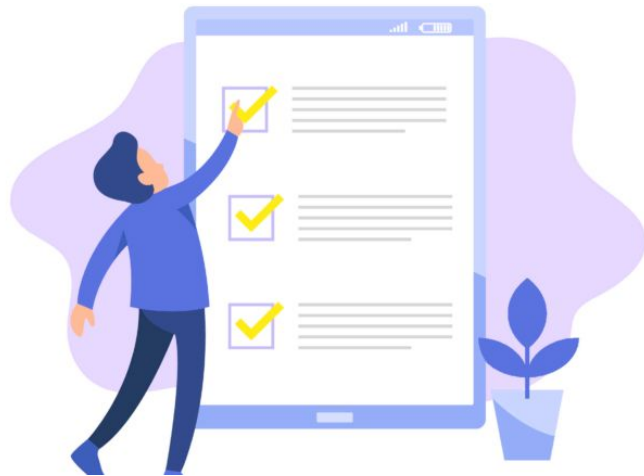
Los pull requests son valiosos en entornos colaborativos, ya que permiten una revisión de código más rigurosa y fomentan la colaboración abierta entre los miembros del equipo.



< > Administrando Pull Request

Crear un Pull Request:

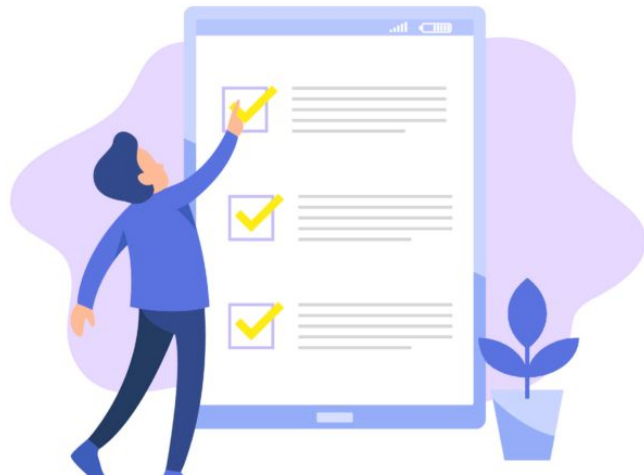
1. Desde la página principal del repositorio en GitHub, haz clic en el botón "New pull request" (Nuevo pull request).
2. Selecciona la rama de la cual deseas fusionar los cambios (rama base) y la rama que contiene los cambios que deseas fusionar (rama comparada).



< > Administrando Pull Request

Crear un Pull Request:

3. Proporciona un título descriptivo y una descripción detallada de los cambios realizados en la rama comparada.
4. Revisa los cambios y, si es necesario, agrega comentarios o realiza modificaciones adicionales.
5. Finalmente, haz clic en el botón "Create pull request" (Crear pull request) para enviar el Pull Request.

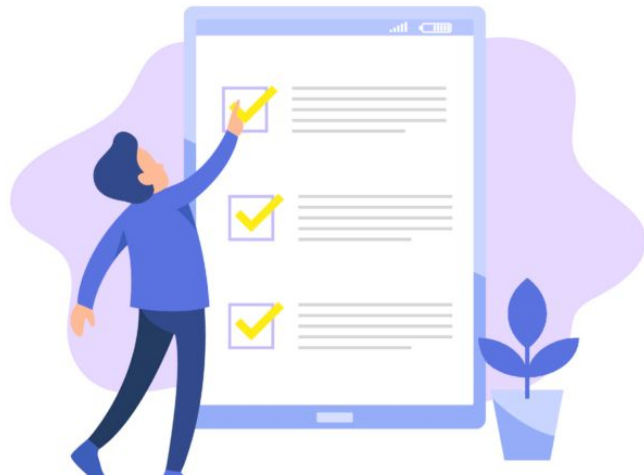


< > Administrando Pull Request

Revisar y comentar en el Pull Request:

Otros colaboradores pueden revisar tus cambios y dejar comentarios en el Pull Request.

Puedes responder a los comentarios, hacer modificaciones adicionales o discutir los cambios propuestos antes de que se fusionen.



< > Administrando Pull Request

Fusionar el Pull Request:

Una vez que los cambios en el Pull Request hayan sido revisados y aprobados, puedes fusionarlos en la rama base.

1. En la página del Pull Request, haz clic en el botón "Merge pull request" (Fusionar pull request) para incorporar los cambios a la rama base.
2. Puedes seleccionar opciones adicionales, como "Squash and merge" (Fusionar y comprimir) o "Rebase and merge" (Fusionar y rebase), dependiendo del flujo de trabajo preferido.
3. Confirma la fusión del Pull Request.
4. Cerrar el Pull Request



< > Administrando Pull Request

Fusionar el Pull Request:

Después de fusionar los cambios, puedes cerrar el Pull Request para indicar que se ha completado.

En la página del Pull Request, haz clic en el botón "**Close pull request** " (Cerrar pull request) para cerrarlo.

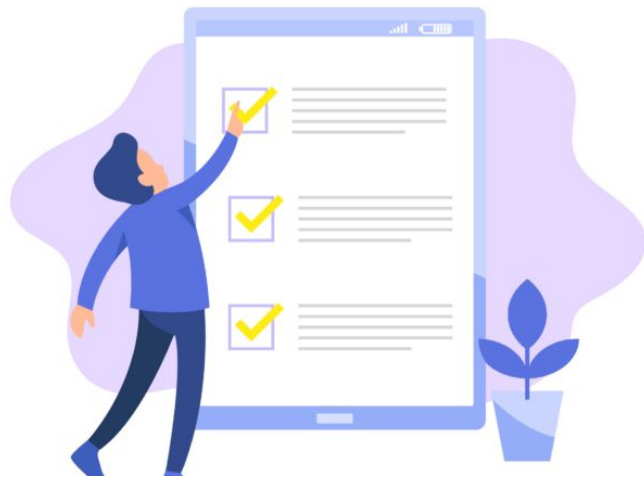
Estos son los pasos básicos para administrar Pull Requests en GitHub. Recuerda que el flujo de trabajo puede variar según las preferencias y políticas del equipo. Además, GitHub proporciona características adicionales, como la posibilidad de asignar revisores, integración continua y otras opciones avanzadas para mejorar la colaboración en el desarrollo de proyectos.



< > Markdown

Markdown es una sintaxis de marcado ligera que se utiliza comúnmente para dar formato y estructurar documentos de texto plano.

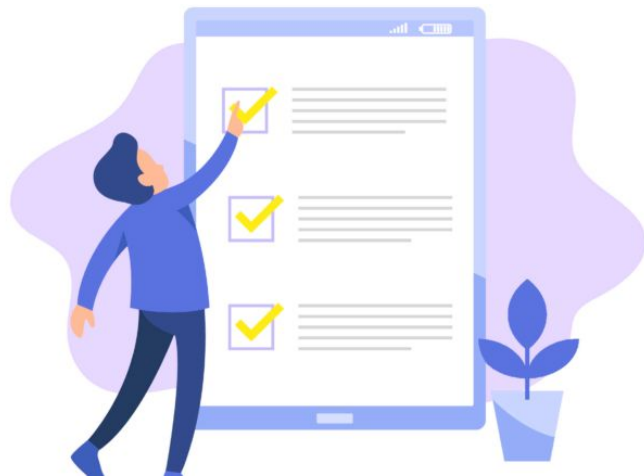
Es ampliamente utilizado en la documentación de proyectos, blogs, foros y plataformas de colaboración como GitHub debido a su simplicidad y facilidad de uso.



< > Markdown

Algunos elementos y características comunes de Markdown:

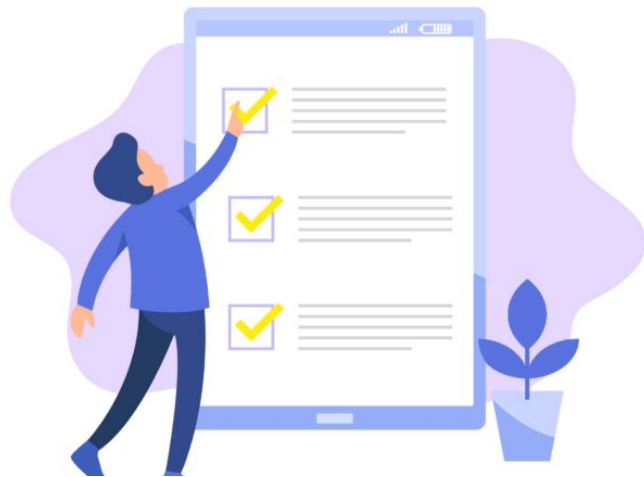
- **Encabezados:** Se pueden crear encabezados utilizando hashtags (#). Cuantos más hashtags uses, más grande será el encabezado (# para encabezado de nivel 1, ## para encabezado de nivel 2, y así sucesivamente).
- **Texto en Negrita y Cursiva:** Puedes enfatizar el texto utilizando asteriscos (*) o guiones bajos (_) alrededor del texto. Dos asteriscos o guiones bajos para negrita, uno para cursiva.



< > Markdown

Algunos elementos y características comunes de Markdown:

- **Listas:** Puedes crear listas ordenadas y no ordenadas utilizando asteriscos, guiones o números.
- **Enlaces:** Los enlaces se crean con el formato [Texto del Enlace](URL).
- **Imágenes:** Las imágenes se insertan de manera similar a los enlaces, pero con un signo de exclamación antes del corchete.
- **Citas:** Las citas se crean utilizando el símbolo mayor que (>). Puedes anidar citas para respuestas múltiples.



Uso de Repositorios Remotos en GitHub

¿En qué consistirá la Demo?

El objetivo es poder generar el primer repositorio de trabajo.

1. Crear una rama y realizar cambios
2. Crear un README.md con Markdown aplicando los distintos elementos y características
3. Subir los cambios a GitHub
4. Crear un Pull Request en GitHub
5. Revisar y comentar en el Pull Request
6. Fusionar el Pull Request:
7. Cerrar el Pull Request

Tiempo: 25 minutos



Ejercicio N° 1

Uso de Push y Pull en GitHub

Uso de Push y Pull en GitHub

Contexto: 🙌

En este ejercicio, aprenderás a trabajar con repositorios remotos en GitHub utilizando los comandos push y pull. Estos comandos son esenciales para mantener sincronizados los cambios entre tu repositorio local y el repositorio remoto en GitHub.

Consigna: 📝

1. Clonar un repositorio existente desde GitHub a tu máquina local.
2. Crear una nueva rama en la que realizarás modificaciones sin afectar la rama principal (main o master).
3. Realizar cambios en un archivo dentro del repositorio clonado.
4. Realizar commits incrementales en la nueva rama, asegurándote de hacer al menos dos commits con diferentes cambios.

Uso de Push y Pull en GitHub

5. Fusionar la nueva rama con main de forma local y resolver posibles conflictos que surjan.
6. Subir los cambios al repositorio remoto usando push.
7. Crear un Pull Request en GitHub y esperar la aprobación de otro colaborador (si es posible).
8. Hacer pull de nuevos cambios del repositorio remoto para actualizar tu repositorio local.
9. Simulación de colaboración:
 - Un compañero hará cambios en el mismo archivo y los subirá al repositorio remoto.
 - Tú harás git pull y resolverás los conflictos en caso de que existan.
 - Confirmar la integración final y verificar que todo está sincronizado correctamente.

Tiempo 🕒: 30 Minutos

¿Alguna consulta?



Resumen

¿Qué logramos en esta clase?

- ✓ **Comprender los fundamentos de GitHub como una plataforma de gestión de control de versiones.**
- ✓ **Dominar la gestión de repositorios remotos y la sincronización de cambios mediante Push y Pull.**
- ✓ **Distinguir entre Fetch y Pull y aplicar el concepto apropiado según la situación.**
- ✓ **Aprender a clonar un repositorio existente para colaborar en proyectos.**
- ✓ **Adquirir habilidades para administrar Pull Requests y revisar y fusionar cambios de colaboradores.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

